เอกสารโครงสร้าง Platform Stack

ระบบตรวจเช็คสภาพเครื่องจักรด้วยความสั่นสะเทือน

Vibration-Based Machine Condition Monitoring System

วิชา:	CS423 Internet of Things and Applications
ผู้สอน:	อาจารย์ทศพร เวชศิริ (tossaporn_w@bu.ac.th)
จัดทำโดย:	ชัชนันท์ สุขยิ่ง 1660706530 พิพัฒน์ ลิขิตวานิช 1660707728 ธนกร เงินยวง 1660708551
สถาบัน:	มหาวิทยาลัยกรุงเทพ คณะเทคโนโลยีสารสนเทศและนวัตกรรม สาขาวิทยาการคอมพิวเตอร์
ภาคการศึกษา:	ภาคการศึกษาที่ 3 / ปีการศึกษา 2569
รูปแบบ:	จำลองด้วย Wokwi Simulator
GitHub:	thanakorn-nger/CS423-vibration-monitor: ระบบตรวจเช็คสภาพเครื่องจักรด้วยเสียง/ความสั่นสะเทือน | CS423 IoT Project

สารบัญ

1. ภาพรวมและ Architecture ของระบบ 3

2. Device Layer 4

2.1 อุปกรณ์และหน้าที่ 4

2.2 ตารางการเชื่อมต่อ Pin (Pin Connection) 4

3. Communication Layer 5

4. Backend / Cloud Layer 5

5. Database Layer 6

6. Frontend / Dashboard Layer 6

7. AI Agent Layer 7

8. Data Flow End-to-End 8

9. MQTT Topic & JSON Payload 9

10. ผลการทดสอบระบบ 10

1. ภาพรวมและโครงสร้างระบบของระบบ (System Architecture)

โครงงานนี้พัฒนาระบบ IoT แบบ End-to-End สำหรับตรวจสอบสภาพเครื่องจักรผ่านการวัดความสั่นสะเทือน โดยใช้ ESP32 เป็น Microcontroller หลักร่วมกับ MPU6050 IMU Sensor ในการวัดแรงสั่น 3 แกน ระบบสามารถตัดสินใจควบคุม Relay อัตโนมัติเมื่อตรวจพบความผิดปกติ และส่งข้อมูลขึ้น Thingsboard Cloud ผ่าน MQTT Protocol เพื่อแสดงผลบน Dashboard แบบ Real-time

Layer	Component	Technology	หน้าที่หลัก
① Device	ESP32 + MPU6050 + Relay + Potentiometer	Arduino C++ / I2C	วัดสั่น ประมวลผล ควบคุม Relay
② Communication	Wi-Fi ⟶ MQTT	PubSubClient / 802.11	ส่ง JSON ขึ้น Cloud ทุก 500ms
③ Backend/Cloud	Thingsboard MQTT Broker	Thingsboard CE	รับข้อมูล Rule Engine REST API
④ Database	Time-Series DB	PostgreSQL (built-in)	เก็บ Telemetry + Alert Log
⑤ Frontend	Thingsboard Dashboard	React Widget	Real-time กราฟ Gauge Alert
⑥ AI Agent	Claude AI (Anthropic) และ Gemini (Google	Claude Sonnet, Gemini Models	ช่วยพัฒนา Code, ออกแบบ Architecture และจัดทำ/เรียบเรียง Docs

Data Flow โดยย่อ: MPU6050 ⟶ ESP32 คำนวณ Vibration ⟶ ควบคุม Relay ⟶ MQTT Publish ⟶ Thingsboard รับ+เก็บ ⟶ Dashboard Real-time

2. Device Layer

2.1 อุปกรณ์และหน้าที่

ระบบใช้อุปกรณ์ทั้งหมด 4 ชิ้นจำลองบน Wokwi Simulator ESP32 ทำหน้าที่ประมวลผลหลักและเชื่อมต่อ Wi-Fi ส่วน MPU6050 ถูกเลือกมาใช้แทน SW-420 Vibration Sensor เนื่องจาก Wokwi ไม่มี SW-420 ในคลัง และ MPU6050 มีความสามารถวัด Acceleration ได้ 3 แกน ด้วยความละเอียด ±2g ซึ่งเหมาะสมกว่า SW-420 ที่เป็นเพียง Digital On/Off Sensor

#	อุปกรณ์	รุ่น / Model	หน้าที่ในระบบ	Protocol
1	Microcontroller	ESP32 DevKit v1	ประมวลผลหลัก Wi-Fi GPIO Control	—
2	Vibration/IMU Sensor (แทน SW-420)	MPU6050	วัดแรงสั่น 3 แกน ±2g (16384 LSB/g)	I2C
3	Relay Module	5V Single Channel	ตัด-ต่อไฟเครื่องจักร (Active LOW)	Digital OUT
4	Potentiometer	Rotary Knob 10kΩ	ปรับ Threshold แบบ Manual (Dynamic)	Analog ADC

2.2 ตารางการเชื่อมต่อ Pin (Pin Connection)

ESP32 ใช้ I2C Bus บน GPIO21 (SDA) และ GPIO22 (SCL) สำหรับ MPU6050 GPIO26 เป็น Digital Output ควบคุม Relay และ GPIO34 เป็น ADC สำหรับ Potentiometer

อุปกรณ์	Pin อุปกรณ์	ESP32 GPIO	ประเภทสัญญาณ	แรงดัน
MPU6050	SDA	GPIO 21	I2C Data	3.3V
MPU6050	SCL	GPIO 22	I2C Clock	3.3V
MPU6050	VCC	3.3V	Power	3.3V
MPU6050	GND	GND	Ground	0V
Relay Module	IN (Signal)	GPIO 26	Digital Output	3.3V
Relay Module	VCC	5V	Power	5V
Relay Module	GND	GND	Ground	0V
Potentiometer	SIG (Wiper)	GPIO 34 (ADC)	Analog Input	3.3V

3. Communication Layer

ESP32 เชื่อมต่อ Wi-Fi ผ่าน SSID "Wokwi-GUEST" ซึ่งเป็น Network จำลองที่ Wokwi จัดให้โดยไม่ต้องใส่ Password หลังจากนั้นจะเชื่อมต่อ MQTT Broker ของ Thingsboard ที่ mqtt.thingsboard.cloud Port 1883 โดยใช้ Access Token เป็น Username ข้อมูลจะถูกส่งในรูปแบบ JSON Payload ทุก 500ms

การเลือกใช้ MQTT เหมาะสมสำหรับ IoT เนื่องจากเป็น Lightweight Protocol ที่ใช้ Bandwidth น้อย รองรับ Publish/Subscribe Pattern และมี QoS Level รองรับความน่าเชื่อถือของข้อมูล

Protocol	รายละเอียด	MQTT Topic / Endpoint
Wi-Fi 802.11	SSID: Wokwi-GUEST (ไม่ใส่ Password)	—
MQTT Publish	ESP32 ⟶ Thingsboard ส่งข้อมูลทุก 500ms	v1/devices/me/telemetry
MQTT Subscribe	Dashboard ⟶ ESP32 ส่งคำสั่งควบคุม Relay	v1/devices/me/rpc/request/+
JSON Format	ส่งด้วย ArduinoJson Library	รายละเอียดใน Section 9

4. Backend / Cloud Layer

โครงงานใช้ Thingsboard Community Edition (CE) ที่ให้บริการบน thingsboard.cloud เป็น Backend หลัก ซึ่งรวม MQTT Broker, REST API, Rule Engine และ Database ไว้ในระบบเดียว ทำให้ไม่ต้องตั้ง Server แยก

การ Provision Device ทำผ่าน Thingsboard Dashboard โดยสร้าง Device ชื่อ "vibration-machine" แล้วนำ Access Token มาใช้เป็น MQTT Username ทุกครั้งที่ ESP32 เชื่อมต่อ

Component	รายละเอียด	Port / Endpoint
MQTT Broker	รับ Telemetry จาก ESP32 ผ่าน Access Token Authentication	1883 (TCP)
Rule Engine	สร้าง Alarm อัตโนมัติเมื่อ vibration > 1.3 พร้อมกำหนด Severity	Internal
REST API	Frontend ดึงข้อมูลย้อนหลังเพื่อแสดง History Chart	/api/v1/...
RPC Service	รับคำสั่งจาก Dashboard ส่งกลับ ESP32 เพื่อควบคุม Relay	MQTT RPC Topic

5. Database Layer

Thingsboard ใช้ PostgreSQL เป็น Database หลักและจัดเก็บข้อมูล Time-Series ในรูปแบบที่เหมาะกับ IoT

ที่มีการเปลี่ยนแปลงบ่อยและต้องการ Query ตาม Timestamp ข้อมูลทุก Field ที่ ESP32 ส่งจะถูกบันทึกโดยอัตโนมัติพร้อม Timestamp

ประเภทข้อมูล	Fields ที่เก็บ	Frequency	Retention
Telemetry (Time-Series)	vibration, smoothVibration, ax, ay, az, status	ทุก 500ms	90 วัน (Default)
Device Attributes	threshold, machineId, lastAlertTime	เมื่อมีการเปลี่ยน	Persistent
Alarm / Alert Log	alarmType, severity, timestamp, triggerValue	เมื่อเกิด Alert	Persistent

6. Frontend / Dashboard Layer

Dashboard พัฒนาบน Thingsboard โดยใช้ Widget ในตัว ผู้ใช้เข้าถึงได้จาก Browser ทุกอุปกรณ์โดยไม่ต้องติดตั้ง App เพิ่มเติม

Dashboard รับข้อมูลจาก Backend ผ่าน WebSocket ทำให้ข้อมูลอัพเดต Real-time โดยไม่ต้อง Refresh หน้าเว็บ

Widget	ประเภท	Key ที่แสดง	ค่า Min / Max
สถานะเครื่องจักร	Cards ⟶ Simple Card	status	—
กราฟ Vibration	Charts ⟶ Time series chart	smoothVibration	—
Gauge วัดแรงสั่น	Gauges ⟶ Radial gauge	vibration	0 – 3
ค่า AX / AY / AZ	Gauges ⟶ Digital gauge	ax, ay, az	-2 – 2
Alert Log	Alarm ⟶ Alarms table	Alarm entity	—

7. AI Agent Layer

ตามที่ได้เรียนในสัปดาห์ที่ 10–12 โครงงานนี้ใช้ Claude AI (Anthropic) เป็น AI Agent ในการช่วยพัฒนาทุกส่วนของระบบ ตั้งแต่การเขียน Code ไปจนถึงการออกแบบ Architecture และจัดทำเอกสาร

ส่วนที่ใช้ AI Agent	AI Tool	สิ่งที่ AI ช่วยทำในโครงงาน
Arduino Code (ESP32)	Claude AI / gemini	เขียน MPU6050 Logic, Exponential Smoothing Filter, Relay State Machine, MQTT Publish
System Architecture	Claude AI	ออกแบบ 6-Layer Architecture, กำหนด MQTT Topic Design และ Data Flow
Thingsboard Config	gemini	ขั้นตอน Device Provisioning, Dashboard Widget Setup, Alarm Rule Config
Debugging	Claude AI / gemini	วิเคราะห์ปัญหา I2C Address, MQTT Connection Error, Threshold Tuning
Documentation	Claude AI / gemini	จัดทำรายงาน, Platform Stack Word/PDF, README สำหรับ GitHub Repository

ข้อจำกัดของ AI Agent ที่พบ

แม้ Claude AI จะช่วยพัฒนาได้รวดเร็ว แต่ยังมีข้อจำกัดที่ต้องระวัง ได้แก่ การแนะนำ Library ที่ไม่รองรับ Wokwi Simulator จึงต้องตรวจสอบ Compatibility ก่อนใช้งานทุกครั้ง AI ไม่สามารถ Run Code จริงได้ จึงต้องทดสอบบน Wokwi เองเสมอ และต้องระบุ Context เช่น Platform หรือ Library Version ให้ชัดเจน เพื่อให้ได้ Code ที่ตรงกับ Use Case

8. Data Flow End-to-End

การไหลของข้อมูลในระบบมี 2 ทิศทาง คือ Uplink (ESP32 ⟶ Cloud ⟶ Dashboard) และ Downlink (Dashboard ⟶ Cloud ⟶ ESP32)

ขั้น	ทิศทาง	การไหลของข้อมูล
①	Uplink	MPU6050 อ่าน Raw Acceleration (int16) ⟶ ESP32 แปลงเป็น g โดยหาร 16384.0
②	Uplink	คำนวณ $\lvert V\rvert = \sqrt{(ax^2+ay^2+az^2)}$ ⟶ EMA Filter (α=0.7) ⟶ เปรียบเทียบกับ Threshold
③	Uplink	ควบคุม Relay (HIGH=เปิด / LOW=ปิด) ตามผลการเปรียบเทียบ พร้อม State Machine
④	Uplink	สร้าง JSON Payload ⟶ MQTT Publish ⟶ Wi-Fi ⟶ mqtt.thingsboard.cloud:1883
⑤	Uplink	Thingsboard Broker รับข้อมูล ⟶ Rule Engine ตรวจ Alert ⟶ บันทึก Time-Series DB
⑥	Uplink	WebSocket Push ⟶ React Dashboard อัพเดต Widget Real-time ทุก 500ms
⑦	Downlink	ผู้ใช้กด Control บน Dashboard ⟶ Thingsboard Publish MQTT RPC Command
⑧	Downlink	ESP32 Subscribe รับ RPC ⟶ ควบคุม Relay ตามคำสั่งจาก Dashboard

9. MQTT Topic & JSON Payload

Thingsboard กำหนด MQTT Topic มาตรฐาน โดย v1/devices/me/telemetry ใช้สำหรับส่งข้อมูล Sensor ขึ้น Cloud ส่วน v1/devices/me/rpc/request/+ ใช้รับคำสั่งควบคุมจาก Dashboard

Field	ชนิดข้อมูล	คำอธิบาย	ตัวอย่างค่า
vibration	float	Vibration Magnitude: $\sqrt{(ax^2+ay^2+az^2)}$	1.054 g
smoothVibration	float	ค่าหลัง EMA Filter (α = 0.7)	1.021 g
ax	float	Acceleration แกน X (หน่วย g)	0.12 g
ay	float	Acceleration แกน Y (หน่วย g)	-0.05 g
az	float	Acceleration แกน Z (หน่วย g)	0.98 g
status	string	สถานะเครื่องจักรปัจจุบัน	"RUNNING" / "STOP"

10. ผลการทดสอบระบบ

การทดสอบแบ่งออกเป็น 2 ระดับ คือ Unit Test (ทดสอบแต่ละ Component) และ Integration Test (ทดสอบการทำงานร่วมกันของทุก Layer) โดยใช้ MQTTX เป็นเครื่องมือทดสอบ MQTT Connection ก่อนที่จะนำโค้ดไปรันบน Wokwi

Test Case	วิธีทดสอบ	ผลที่คาดหวัง	ผลจริง
MQTT Connection (MQTTX)	ส่ง Test JSON ผ่าน MQTTX ไป Thingsboard	Latest telemetry แสดงข้อมูล	✅ ผ่าน
MPU6050 Init	เปิด Wokwi ดู Serial Monitor	MPU6050 Connected	✅ ผ่าน
Normal Vibration (< 1.3g)	ค่า Default ของ Wokwi Simulator	Relay HIGH, RUNNING	✅ ผ่าน
Abnormal Vibration (> 1.3g)	ปรับ MPU6050 ให้สั่นเกิน Threshold	Relay LOW, STOP	✅ ผ่าน
Recovery	ลดค่าสั่นกลับมาต่ำกว่า Threshold	Relay HIGH, RUNNING	✅ ผ่าน
Dashboard Real-time	ดู Thingsboard Dashboard ขณะ Wokwi รัน	Widget อัพเดตทุก 500ms	✅ ผ่าน
Alert Trigger	สั่นเกิน Threshold ⟶ ดู Alarm Widget	Alarm Widget แสดง Alert	✅ ผ่าน
MQTT Auto-Reconnect	รีสตาร์ท Wokwi แล้วดู Serial Monitor	Reconnect อัตโนมัติ	✅ ผ่าน

ระบบผ่านการทดสอบทั้ง 8 Test Case ครบถ้วน แสดงให้เห็นว่าระบบ IoT แบบ End-to-End ทำงานได้ถูกต้องตาม Requirement ทั้งในส่วน Hardware (ESP32 + MPU6050 + Relay), Communication (MQTT), Backend (Thingsboard) และ Frontend (Dashboard Real-time)